UNIT IV

Searching Techniques, Hashing and Sorting Techniques

Theory notes for the written exam

Part	Topics
A. Searching	Linear search, binary search
B. Hashing	Direct-address tables, hash tables, open addressing, perfect hashing
C. Sorting	Bubble sort, selection sort, insertion sort, merge sort, time complexity analysis

PART A: SEARCHING TECHNIQUES

1. Introduction to Searching

Definition: Searching is the process of finding the location of a given value (called the **key**) in a collection of data such as an array.

- If the key is found, the search is **successful** and its position is returned.
- If the key is not present, the search is **unsuccessful**.
- The two basic searching techniques are **linear search** and **binary search**.

2. Linear Search

Definition: Linear search (sequential search) is a method in which the key is compared with every element of the list **one by one, from the first element**, until the key is found or the list ends.

Algorithm: Linear Search

1. Start from the first element of the array (index 0).
2. Compare the key with the current element.
3. If they are equal, the search is successful. Return the position and stop.
4. If they are not equal, move to the next element.
5. Repeat steps 2 to 4 until the end of the array.
6. If the end is reached without a match, the key is not present (unsuccessful search).

Example: Array = 10, 25, 30, 45, 60. Search for 45: it is compared with 10, 25, 30 and 45. Found at index 3 after **4 comparisons**. Search for 99: compared with all 5 elements, not found after **5 comparisons**.

Time complexity

Case	When	Comparisons	Time
Best case	Key is the first element	1	O(1)
Average case	Key is somewhere in the middle	about n/2	O(n)
Worst case	Key is the last element or is absent	n	O(n)

Space complexity: O(1) (no extra memory is needed).

Advantages	Disadvantages
Very simple to understand and use	Slow for large lists; time grows with n
Works on **unsorted** data	Compares many elements unnecessarily
Works on arrays and linked lists	Not suitable when searches are frequent
No extra memory and no preparation needed	Binary search is much faster on sorted data

3. Binary Search

Definition: Binary search is a searching method for a **sorted** list in which the key is compared with the **middle element**. If they are not equal, one half of the list is **discarded** and the search continues in the other half. This is repeated until the key is found or no elements are left.

Condition: the array must be **sorted**, and direct access to the middle element must be possible (so it works on arrays, not on linked lists).

Algorithm: Binary Search
1. Set low = 0 and high = n - 1.
2. Repeat steps 3 to 6 while low <= high.
3. Find the middle position: mid = (low + high) / 2 (integer part).
4. If A[mid] = key, the search is successful. Return mid and stop.
5. If key < A[mid], search the left half: set high = mid - 1.
6. If key > A[mid], search the right half: set low = mid + 1.
7. If low becomes greater than high, the key is not present (unsuccessful search).

Example 1 (successful): Array = 10, 20, 30, 40, 50, 60, 70 (indexes 0 to 6). Key = 60.

low	high	mid	A[mid]	Action
0	6	3	40	key is larger: low = 4
4	6	5	60	equal: found at index 5

Found in **2 comparisons** (linear search would need 6).

Example 2 (unsuccessful): Same array, key = 35.

low	high	mid	A[mid]	Action
0	6	3	40	key is smaller: high = 2
0	2	1	20	key is larger: low = 2
2	2	2	30	key is larger: low = 3

Now low (3) is greater than high (2), so 35 is **not present**.

Time complexity

Each comparison halves the list: n, n/2, n/4, ..., 1. The number of halvings is $\log_2 n$, so at most about $\mathbf{\log_2 n + 1}$ comparisons are needed. For n = 1000 this is only about 10 comparisons.

Case	When	Time
Best case	Key is the middle element (found at the first comparison)	O(1)
Average case	Key found after several halvings	O(log n)
Worst case	Key is absent or found in the last step	O(log n)

Space complexity: O(1) for the iterative method; O(log n) for the recursive method (stack space).

Advantages	Disadvantages
Very fast: O(log n) time	The list must be **sorted** first
Needs very few comparisons for large data	Cannot be used directly on linked lists (no direct access to the middle)
No extra memory (iterative)	Sorting cost is added if the data is unsorted
Good when many searches are done on fixed data	Insertion and deletion are costly because the order must be kept

4. Linear Search vs Binary Search

Point	Linear search	Binary search
Data condition	Works on sorted and unsorted data	Works only on **sorted** data
Method	Checks elements one by one	Compares with the middle and discards half
Best case	O(1)	O(1)
Average / worst case	O(n)	O(log n)
Speed for large data	Slow	Very fast
Data structure	Array or linked list	Array (direct access needed)
Implementation	Very simple	Slightly complex
Extra memory	O(1)	O(1) iterative, O(log n) recursive

PART B: HASHING

5. Introduction to Hashing

Definition: Hashing is a technique in which the position of a key in a table is **calculated from the key itself** by a **hash function**, so that insertion, searching and deletion take **O(1) time on average**.

Term	Meaning
Hash function h(k)	Converts a key k into a slot number (index) of the table.
Hash table	An array of m slots in which the keys are stored.
Collision	Two different keys give the same slot: h(k1) = h(k2).
Load factor (alpha)	alpha = n / m, where n = number of keys stored and m = table size.

6. Direct-Address Tables

Definition: A **direct-address table** is an array T[0 .. M-1] in which the **key itself is the index**: key k is stored in T[k]. It is used when the keys are small integers taken from a small universe {0, 1, ..., M-1}.

Algorithm: Operations on a direct-address table
Insert key k: store the key (or its record) in T[k].
Search key k: look at T[k]. If it is filled, the key is present; if empty, it is absent.
Delete key k: make T[k] empty.

All operations take **O(1) time in the worst case**, and there are no collisions.

Advantages	Disadvantages
Very simple	Needs one slot for every possible key; if the universe is large, memory is wasted (10-digit phone numbers need 10 billion slots)
O(1) worst case for all operations	Works only for integer keys in a small range

7. Hash Tables

Definition: A **hash table** stores keys in a table of size m (much smaller than the universe of keys) using a hash function h that maps every key to a slot from 0 to m-1.

A good hash function is fast to calculate, gives the same value for the same key, spreads the keys evenly, and causes few collisions.

Hash functions

- **Division method:** h(k) = k mod m. Choose m as a **prime number**, not a power of 2. Example: m = 11, h(54) = 54 mod 11 = 10.
- **Multiplication method:** h(k) = floor(m x (kA mod 1)), with 0 < A < 1 (A = 0.618 is a good value). Example: k = 21, m = 10: 21 x 0.618 = 12.978, fractional part 0.978, 10 x 0.978 = 9.78, so h(21) = 9.

Collision

Definition: A **collision** occurs when two different keys hash to the same slot. It cannot be avoided because there are more possible keys than slots. It is handled by **chaining** or by **open addressing**.

Collision resolution by chaining

Definition: In **chaining** every slot holds a **linked list**; all keys that hash to the same slot are stored in the list of that slot.

Algorithm: Chaining
Insert key k: compute h(k) and add the key at the head of the list in slot h(k).
Search key k: compute h(k) and search the list in that slot for the key.
Delete key k: compute h(k), find the key in the list and remove its node.

Example: m = 7, h(k) = k mod 7, keys 50, 700, 76, 85, 92, 73, 101.

Slot	0	1	2	3	4	5	6
Keys	700	50, 85, 92	empty	73, 101	empty	empty	76

Analysis: insert O(1); search O(1 + alpha) on average; worst case O(n) when all keys fall in one list. alpha can be more than 1.

Advantages	Disadvantages
Simple; the table never gets full; deletion is easy	Extra memory for pointers; lists become long if the hash function is poor

8. Open Addressing

Definition: In **open addressing** all the keys are stored **inside the table itself** (no linked lists). If a slot is occupied, other slots are tried one by one according to a fixed rule called **probing**. So n <= m and alpha <= 1.

Algorithm: Insert (open addressing)
1. Set i = 0.
2. Calculate j = h(k, i).
3. If slot j is empty (or marked DELETED), store the key there and stop.
4. Otherwise set i = i + 1 and repeat from step 2.
5. If i reaches m, the table is full (overflow).

Algorithm: Search (open addressing)
1. Set i = 0.
2. Calculate j = h(k, i).
3. If slot j contains the key, the search is successful; stop.
4. If slot j is empty, the key is not present; stop.
5. Otherwise set i = i + 1 and repeat from step 2 (until i = m).

Algorithm: Delete (open addressing)
1. Search for the key as above.
2. If found, do **not** make the slot empty; mark it **DELETED**.

Why DELETED? If the slot were emptied, a search for another key that was placed further along the same probe sequence would stop at this empty slot and wrongly say 'not found'.

Example used below: keys 89, 18, 49, 58, 69.

(a) Linear probing

Definition: The next slots are tried one after another: **h(k, i) = (h'(k) + i) mod m**, for i = 0, 1, 2, ... The search wraps around at the end of the table.

m = 10, h'(k) = k mod 10. 89 goes to slot 9 and 18 to slot 8. For 49, slot 9 is taken, so slot 0. For 58, slots 8, 9, 0 are taken, so slot 1. For 69, slots 9, 0, 1 are taken, so slot 2.

0	1	2	3	4	5	6	7	8	9
49	58	69						18	89

- **Advantage:** simplest to implement.
- **Disadvantage: primary clustering**: filled slots join into long runs that keep growing and slow down the search.

(b) Quadratic probing

Definition: The gap between the slots tried grows as a square: **h(k, i) = (h'(k) + c1 i + c2 i^2) mod m**. In the simple form the slots tried are h', h'+1, h'+4, h'+9, ...

m = 10, h(k, i) = (k mod 10 + i^2) mod 10. For 49: slot 9 is taken, so 9+1 = 0. For 58: 8 is taken, 9 is taken, 8+4 = 12 gives slot 2. For 69: 9 is taken, 0 is taken, 9+4 = 13 gives slot 3.

0	1	2	3	4	5	6	7	8	9
49		58	69					18	89

- **Advantage:** removes primary clustering.
- **Disadvantage: secondary clustering** (keys with the same starting slot follow the same path); may not reach every slot. If m is prime and the table is less than half full, an empty slot is always found.

(c) Double hashing

Definition: Two hash functions are used and the second gives the step size: **h(k, i) = (h1(k) + i x h2(k)) mod m**. h2(k) must never be 0 and should be relatively prime to m (take m prime).

m = 11, h1(k) = k mod 11, h2(k) = 1 + (k mod 10). For 69: h1 = 3 is taken by 58; h2 = 1 + 9 = 10; next slot = (3 + 10) mod 11 = 2.

0	1	2	3	4	5	6	7	8	9	10
	89	69	58		49		18			

- **Advantage:** almost no clustering; best performance of the three.
- **Disadvantage:** two hash functions must be calculated, so each probe is slower.

Comparison of probing methods

Point	Linear	Quadratic	Double hashing
Step size	1	Increases as a square	Depends on the key
Primary clustering	Yes	No	No
Secondary clustering	Yes	Yes	No
Performance	Poorest	Better	Best

Performance: for an unsuccessful search the expected number of probes is at most 1 / (1 - alpha), so the table should not be allowed to become nearly full.

Chaining vs open addressing

Point	Chaining	Open addressing
Keys stored	In linked lists outside the table	Inside the table
Load factor	Can be more than 1	At most 1
Extra memory	Needed for pointers	Not needed
Deletion	Easy	Difficult (DELETED mark)
Clustering	None	Can occur

9. Perfect Hashing

Definition: Perfect hashing is a hashing technique that gives **O(1) search time in the worst case** with **no collisions**. It works only for a **static set of keys**, that is, keys known in advance with no insertions or deletions later.

Algorithm: Perfect hashing (two-level scheme)
Level 1: hash the n keys into a table of size m = n using a hash function h. Slot j receives n_j keys.
Level 2: for every slot j create a **secondary table of size n_j^2** with its own hash function h_j, chosen so that there is **no collision**. If a collision occurs, choose another function and try again.
Search key k: compute j = h(k), then compute $h_j(k)$ and look at that position in the secondary

table of slot j.

Only two hash calculations are needed, so the worst case search time is O(1).

Why size n_j^2? In a table of this size the chance of any collision is less than 1/2, so a good function is found in about 2 tries. The total space is **O(n)**. The hash functions are chosen at random from a **universal family** (two different keys collide with probability at most 1/m).

Example: keys 12, 25, 31, 44; m = 4; h(k) = k mod 4. Hash values: 12 gives 0, 25 gives 1, 31 gives 3, 44 gives 0. Slot 0 has 2 keys, so its secondary table has size $2^2 = 4$. With $h_0(k) = (k \bmod 5) \bmod 4$: $h_0(12) = 2$ and $h_0(44) = 0$, so there is no collision. Slots 1 and 3 have one key each (table size 1).

Advantages	Disadvantages
O(1) search in the worst case; space O(n)	Only for static keys; no insertion or deletion later; more complex to build

Used for: compiler reserved-word tables, fixed dictionaries, read-only databases.

10. Hashing: Summary

Method	Idea	Search time	Limitation
Direct-address table	Key is the index	O(1) worst case	Slot for every possible key
Chaining	Linked list in each slot	O(1 + alpha) average	Extra memory
Open addressing	Probe other slots in the table	1/(1 - alpha) probes	alpha at most 1; clustering
Perfect hashing	Two levels, no collision	O(1) worst case	Static keys only

PART C: SORTING TECHNIQUES

11. Introduction to Sorting

Definition: Sorting is the process of arranging the elements of a list in a particular order, either **ascending** (smallest to largest) or **descending** (largest to smallest).

Sorting makes searching faster (for example binary search needs sorted data) and makes data easy to understand and process.

Important terms

Term	Meaning
Stable sort	A sort that keeps the **original order of equal elements**. (If two elements have the same value, the one that came first stays first.)
In-place sort	A sort that needs only a constant amount of extra memory, O(1).
Adaptive sort	A sort that runs faster when the input is already (nearly) sorted.
Internal sort	All the data is held in main memory during sorting.
External sort	The data is too large for memory and is kept on disk (merge sort is the basis).
Pass	One complete scan through the list during sorting.

Time complexity analysis

The **time complexity** of a sorting method tells how the number of steps (mainly comparisons and swaps) grows with the number of elements n. It is stated for the **best case** (most favourable input), **average case** and **worst case** (least favourable input), using big-O notation.

12. Bubble Sort

Definition: Bubble sort is a simple sorting method that repeatedly compares **adjacent elements** and **swaps** them if they are in the wrong order. After each pass the largest remaining element moves ('bubbles') to its correct place at the end.

Algorithm: Bubble Sort
1. Repeat steps 2 and 3 for pass = 1 to n - 1.
2. Compare each pair of adjacent elements, from the first element up to the end of the unsorted part.
3. If the left element is greater than the right element, swap them.
4. After each pass the largest element is placed at its final position.
5. (Improved version) If no swap happens in a pass, the list is already sorted: stop.

Example: Sort 5, 1, 4, 2, 8 in ascending order.

Pass	Array after the pass	Swaps
1	1 4 2 5 8	3
2	1 2 4 5 8	1
3	1 2 4 5 8	0

In pass 1 the largest element 8 is already at the end. In pass 3 no swap happens, so the improved version stops: the list is sorted.

Time complexity

Pass 1 makes n-1 comparisons, pass 2 makes n-2, and so on. Total comparisons = (n-1) + (n-2) + ... + 1 = **n(n-1)/2**, which is $O(n^2)$.

Case	Time	Reason
Best case	O(n)	List already sorted; one pass with no swap (improved version)
Average case	$O(n^2)$	About n(n-1)/4 swaps
Worst case	$O(n^2)$	List in reverse order; n(n-1)/2 comparisons and swaps

Space: O(1) (in-place). **Stable:** Yes.

Advantages	Disadvantages
Very simple to understand and implement	Very slow for large lists: $O(n^2)$
In-place: no extra memory	Many unnecessary swaps
Stable	Rarely used in practice
Detects a sorted list early (improved version)	Even slower than insertion sort in most cases

13. Selection Sort

Definition: Selection sort repeatedly **selects the smallest element** from the unsorted part of the list and **swaps** it with the first element of that unsorted part. The sorted part grows from the left one element at a time.

Algorithm: Selection Sort
1. Repeat steps 2 to 4 for i = 0 to n - 2.
2. Assume that A[i] is the smallest element.
3. Scan the remaining elements A[i+1] to A[n-1] and find the position of the smallest element.
4. Swap the smallest element with A[i].

Example: Sort 64, 25, 12, 22, 11.

Pass	Smallest found	Array after the pass
1	11	11 25 12 22 64
2	12	11 12 25 22 64
3	22	11 12 22 25 64
4	25	11 12 22 25 64

Time complexity

To find the minimum in each pass, all remaining elements are compared, whatever the input. Comparisons = (n-1) + (n-2) + ... + 1 = **n(n-1)/2**. The number of swaps is at most n - 1.

Case	Time
Best case	$O(n^2)$
Average case	$O(n^2)$
Worst case	$O(n^2)$

Space: O(1) (in-place). **Stable:** No (the swap can change the order of equal elements; for example 4a, 4b, 3 becomes 3, 4b, 4a).

Advantages	Disadvantages
Simple to implement	Always $O(n^2)$, even for a sorted list (not adaptive)
Makes the **least number of swaps** (at most n - 1), useful when writing to memory is costly	Not stable
In-place: no extra memory	Slow for large lists

14. Insertion Sort

Definition: Insertion sort builds the sorted list one element at a time. Each new element is taken and **inserted into its correct position** in the already sorted part on its left, by shifting the larger elements one place to the right (like arranging playing cards in the hand).

Algorithm: Insertion Sort
1. Treat the first element as a sorted part of size 1.
2. For i = 1 to n - 1, take the element A[i] as the key.
3. Compare the key with the elements of the sorted part, moving from right to left.
4. Shift every element that is greater than the key one position to the right.
5. Place the key in the gap that is created.

Example: Sort 12, 11, 13, 5, 6.

i	Key	Array after inserting the key
1	11	11 12 13 5 6
2	13	11 12 13 5 6
3	5	5 11 12 13 6
4	6	5 6 11 12 13

Time complexity

Case	Time	Reason
Best case	O(n)	List already sorted: only n-1 comparisons, no shifting
Average case	$O(n^2)$	About half of the sorted part is shifted each time
Worst case	$O(n^2)$	List in reverse order: 1 + 2 + ... + (n-1) = n(n-1)/2 shifts

Space: O(1) (in-place). **Stable:** Yes. **Adaptive:** Yes.

Advantages	Disadvantages
Simple to implement	$O(n^2)$ for large random data
Very fast for small lists and **nearly sorted** lists	Many shifts when the list is in reverse order
In-place, stable and adaptive	Slower than merge sort for large lists
Can sort a list as it is received (online)	

15. Merge Sort

Definition: Merge sort is a sorting method based on **divide and conquer**. The list is divided into two halves, each half is sorted (by the same method, recursively), and then the two sorted halves are **merged** into one sorted list.

Algorithm: Merge Sort (A, low, high)
1. If low >= high, the list has one element and is already sorted; stop.
2. **Divide:** find the middle, mid = (low + high) / 2.
3. **Conquer:** sort the left half A[low .. mid] by merge sort.
4. **Conquer:** sort the right half A[mid+1 .. high] by merge sort.
5. **Combine:** merge the two sorted halves into one sorted list.

Algorithm: Merge (two sorted halves)
1. Place one pointer at the start of each sorted half.
2. Compare the two elements pointed to, copy the **smaller** one into the result and move that pointer forward.
3. Repeat step 2 until one half is finished.
4. Copy all the remaining elements of the other half into the result.
5. Copy the result back into the array.

Example: Sort 38, 27, 43, 3, 9, 82, 10, 15. The list is divided until every part has one element, then the parts are merged back in sorted order.

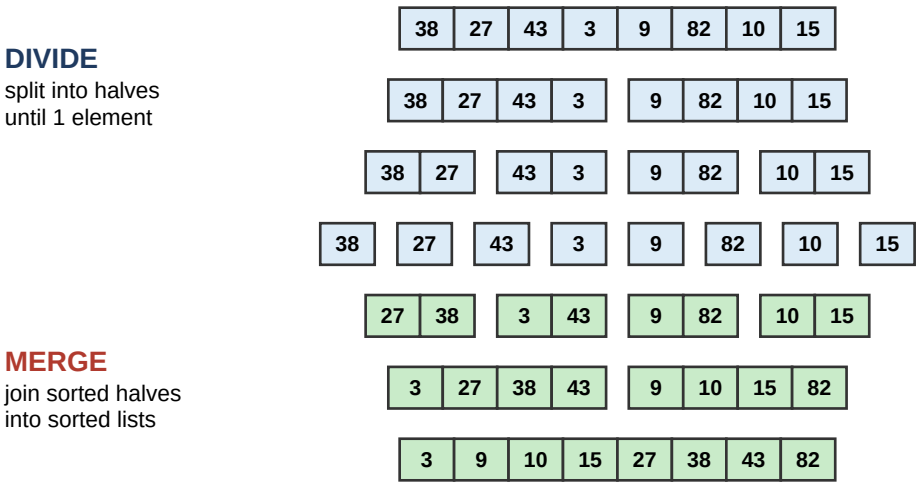

Figure 1: Merge sort: divide into halves, then merge sorted halves

Merging example: merge the sorted halves 3, 27, 38, 43 and 9, 10, 15, 82. Compare 3 and 9: take 3. Compare 27 and 9: take 9. Then 27 and 10: take 10. Then 27 and 15: take 15. Then 27 and 82: take 27; next 38: take 38; next 43: take 43. Left half is finished, so copy 82. Result: 3, 9, 10, 15, 27, 38, 43, 82.

Time complexity

The list is halved repeatedly, so there are $\log_2 n$ levels of division. At every level the merging of all parts together takes about **n** steps. So total time = n x $\log_2$ n. As a recurrence: T(n) = 2 T(n/2) + n, which gives **O(n log n)**.

Case	Time
Best case	O(n log n)
Average case	O(n log n)
Worst case	O(n log n)

Space: O(n) extra memory for merging (not in-place). **Stable:** Yes.

Advantages	Disadvantages
Always O(n log n), even in the worst case	Needs O(n) extra memory
Stable	More complex than the simple sorts
Good for large lists, linked lists and external sorting	Slower than simple sorts for very small lists (recursion overhead)
Performance does not depend on the input order	Not in-place

16. Comparison of Sorting Methods

Sort	Best	Average	Worst	Extra space	Stable	In-place	Technique
Bubble	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes	Swap adjacent elements
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Yes	Select minimum and swap
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Yes	Insert into sorted part
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No	Divide and conquer

Which sort to use?

- **Small or nearly sorted data:** insertion sort.
- **Large data, guaranteed speed and stability:** merge sort.
- **When writing to memory is costly (fewest swaps):** selection sort.
- **For learning the idea of sorting:** bubble sort.

Searching complexities at a glance

Method	Best	Average	Worst	Needs sorted data?
Linear search	O(1)	O(n)	O(n)	No
Binary search	O(1)	O(log n)	O(log n)	Yes

17. Important Questions

1. What is searching? Explain linear search with an algorithm and an example. (5 marks)
2. Explain binary search with an algorithm and an example. Why must the data be sorted? (5-10 marks)
3. Differentiate between linear search and binary search. (5 marks)
4. Derive the time complexity of binary search. (3-5 marks)
5. What is hashing? Define hash function, hash table, collision and load factor. (5 marks)
6. Explain direct-address tables with advantages and disadvantages. (5 marks)
7. What is a collision? Explain chaining with an example. (5 marks)
8. Explain open addressing with insert, search and delete. Why is a slot marked DELETED? (5 marks)
9. Explain linear probing, quadratic probing and double hashing with examples. (10 marks)
10. What is perfect hashing? Explain the two-level scheme with an example. (5-10 marks)
11. What is sorting? Explain stable sort, in-place sort and time complexity analysis. (5 marks)
12. Explain bubble sort with an algorithm and a worked example. (5-10 marks)
13. Explain selection sort with an algorithm and a worked example. (5-10 marks)
14. Explain insertion sort with an algorithm and a worked example. (5-10 marks)
15. Explain merge sort with an algorithm, an example and its time complexity. (10 marks)
16. Compare bubble, selection, insertion and merge sort. (5 marks)